Отлично, вижу твой энтузиазм! 👍 Теперь, когда у нас есть простой график, давай научимся делать его **более информативным и красивым**! Ведь просто линия на пустом фоне не всегда достаточно понятна.

Настройка элементов графика в Matplotlib – это как добавление деталей и красок к твоему рисунку. Мы можем добавить:

- **Заголовок графика (Title):** Чтобы сказать зрителю, что именно изображено на графике.
- **Подписи к осям (Axis labels):** Чтобы указать, какие величины отложены по осям X и Y, и в каких единицах измерения.
- **Цвет и стиль линии (Line color and style):** Чтобы сделать график более выразительным и различимым.

1. Добавление заголовка и подписей осей:

Matplotlib предоставляет простые функции для добавления заголовка и подписей к осям:

- `plt.title('Заголовок графика')` **:** Добавляет **заголовок** к текущему графику. В кавычках нужно указать текст заголовка.
- `plt.xlabel('Подпись оси X')` **:** Добавляет **подпись к оси X**. В кавычках нужно указать текст подписи.
- `plt.ylabel('Подпись оси Y')` **:** Добавляет **подпись к оси Y**. В кавычках нужно указать текст подписи.

Пример:

Возьмем наш предыдущий код и добавим заголовок и подписи к осям:

```
import matplotlib.pyplot as plt

x_values = [1, 2, 3, 4, 5]
y_values = [2, 4, 1, 3, 5]

plt.plot(x_values, y_values)

plt.title('Зависимость Y от X') # Добавляем заголовок
plt.xlabel('Значения X') # Добавляем подпись к оси X
plt.ylabel('Значения Y') # Добавляем подпись к оси Y

plt.show()
```

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Что изменилось? Теперь у графика должен быть заголовок "Зависимость Y от X", подпись "Значения X" под осью X и подпись "Значения Y" слева от оси Y. График стал гораздо понятнее!

2. Настройка цвета и стиля линии:

Функция `plt.plot()` позволяет настраивать **цвет и стиль линии** прямо в момент построения графика, с помощью **дополнительных аргументов**.

- **`color='цвет'`:** Задает **цвет линии**. Цвет можно задавать разными способами, но пока давай использовать **названия цветов** (например, `'red'`, `'blue'`, `'green'`, `'black'`, `'purple'`, `'orange'`, `'cyan'`, `'magenta'`, `'yellow'`, `'lime'`, `'teal'`, `'olive'`, `'maroon'`, `'navy'`, `'fuchsia'`, `'silver'`, `'gray'`).
- **`linestyle='стиль'` или сокращенно `ls='стиль'`:** Задает **стиль линии**. Основные стили:
  - `'-'` или `'solid'` – **сплошная линия** (по умолчанию).
  - `'--'` или `'dashed'` – **пунктирная линия**.
  - `':'` или `'dotted'` – **точечная линия**.
  - `'-.'` или `'dashdot'` – **штрих-пунктирная линия**.
  - `''` или `'None'` – **без линии** (только точки, если есть маркеры).

Пример:

Давай изменим цвет линии на красный и сделаем ее пунктирной:

```
import matplotlib.pyplot as plt

x_values = [1, 2, 3, 4, 5]
y_values = [2, 4, 1, 3, 5]

plt.plot(x_values, y_values, color='red', linestyle='--') # Задаем цвет 'red' и стиль '--'

plt.title('Зависимость Y от X (красная пунктирная линия)')
plt.xlabel('Значения X')
plt.ylabel('Значения Y')

plt.show()
```

Задание:

1. **Запусти этот код.**

2. **Посмотри на результат.** Теперь линия графика должна быть **красной и пунктирной**. Попробуй менять значения `color` и `linestyle` на другие, чтобы поэкспериментировать с разными вариантами. Например:
   - `color='blue', linestyle=':'` (синяя точечная линия)
   - `color='green', linestyle='-.'` (зеленая штрих-пунктирная линия)
   - `color='purple', linestyle='-'` (фиолетовая сплошная линия)

Практическое задание:

1. **Возьми код своего первого графика (`plt.plot(x_values, y_values)`).**
2. **Добавь к нему:**
   - Заголовок графика на русском языке (любой, подходящий по смыслу).
   - Подпись к оси X "Ось абсцисс".
   - Подпись к оси Y "Ось ординат".
   - Измени цвет линии на **зеленый**.
   - Сделай линию **штрих-пунктирной**.

Запусти код и убедись, что все изменения применились к графику. Поэкспериментируй с разными цветами и стилями линий.

После выполнения задания, обсудим результаты и перейдем к следующей теме - **точечным диаграммам (scatter plots)**. Как тебе такой план? И есть ли вопросы по настройке заголовка, подписей осей, цвета и стиля линии? Не стесняйся спрашивать! 😉

Отлично! Раз ты готов двигаться дальше, то переходим к **точечным диаграммам (scatter plots)**! 📌

Точечные диаграммы – это еще один очень важный и распространенный тип графиков в визуализации данных. В отличие от линейных графиков, которые показывают **линию тренда** или **зависимость** между переменными, **точечные диаграммы показывают отдельные точки данных**.

Когда используются точечные диаграммы?

- **Для исследования взаимосвязи между двумя переменными:** Точечные диаграммы отлично подходят для визуализации того, как одна переменная меняется в зависимости от другой. Можно увидеть, есть ли какая-то закономерность, тренд, или точки распределены случайно.

- **Для выявления кластеров и выбросов:** На точечной диаграмме часто хорошо видны группы точек (кластеры) и точки, которые сильно отличаются от остальных (выбросы).
- **Когда важны отдельные точки данных:** В отличие от линейных графиков, где линия как бы "сглаживает" данные, точечные диаграммы показывают каждую точку данных индивидуально, что может быть важно, если каждая точка представляет собой отдельное наблюдение или событие.
- **Для визуализации многомерных данных (в сочетании с цветом и размером точек):** Точечные диаграммы можно использовать для отображения не только двух, но и большего количества переменных, если использовать цвет, размер и форму точек для кодирования дополнительных измерений.

Как создать точечную диаграмму в Matplotlib?

Для создания точечных диаграмм в Matplotlib используется функция `plt.scatter()` .

Основной синтаксис `plt.scatter(x, y, ...)` :

- `x` – **Данные для оси X** (как и в `plt.plot()`). Это может быть список, NumPy массив или Pandas Series.
- `y` – **Данные для оси Y** (как и в `plt.plot()`). Должны быть той же длины, что и `x` .
- **Дополнительные аргументы (параметры):** Позволяют настроить внешний вид точек: цвет, размер, форму маркера и т.д. Мы рассмотрим некоторые из них ниже.

Простейшая точечная диаграмма:

Возьмем те же данные `x_values` и `y_values` , что и для линейного графика, и построим точечную диаграмму:

```
import matplotlib.pyplot as plt

x_values = [1, 2, 3, 4, 5]
y_values = [2, 4, 1, 3, 5]

plt.scatter(x_values, y_values) # Функция scatter() строит точечную диаграмму

plt.title('Точечная диаграмма зависимости Y от X')
plt.xlabel('Значения X')
plt.ylabel('Значения Y')

plt.show()
```

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Что ты видишь? Теперь вместо линии ты должен увидеть **набор точек**, расположенных в соответствии с координатами (x, y). Форма точек по умолчанию – кружки.

Настройка внешнего вида точек в `plt.scatter()`:

Функция `plt.scatter()` предоставляет множество параметров для настройки внешнего вида точек. Рассмотрим основные:

- **`marker='маркер'`:** Задает **форму маркера** (точки). Основные маркеры:
  - `'o'` – **кружок** (по умолчанию).
  - `'.'` – **точка**.
  - `','` – **пиксель**.
  - `'x'` – **крестик**.
  - `'+'` – **плюс**.
  - `'*'` – **звездочка**.
  - `'s'` – **квадрат**.
  - `'d'` – **ромб**.
  - `'^'` – **треугольник вверх**.
  - `'v'` – **треугольник вниз**.
  - `<` – **треугольник влево**.
  - `'>'` – **треугольник вправо**.
  - `'p'` – **пятиугольник**.
  - `'h'` – **шестиугольник 1**.
  - `'H'` – **шестиугольник 2**.
- **`s=размер`:** Задает **размер маркера** в **пунктах в квадрате**. Чем больше число, тем больше размер точек.
- **`c=цвет` или `color=цвет`:** Задает **цвет маркера**. Цвет можно задавать так же, как и для линий в `plt.plot()` (названия цветов, HEX-коды, RGB-значения и т.д.).

Пример:

Давай изменим форму маркера на крестик, увеличим размер точек и сделаем их зелеными:

```python
import matplotlib.pyplot as plt

x_values = [1, 2, 3, 4, 5]
y_values = [2, 4, 1, 3, 5]

plt.scatter(x_values, y_values, marker='x', s=100, c='green') # Задаем маркер 'x', размер 100, цв

plt.title('Точечная диаграмма (крестики, зеленые, большие)')
plt.xlabel('Значения X')
plt.ylabel('Значения Y')

plt.show()
```

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Теперь точки должны быть в виде **зеленых крестиков** большего размера. Попробуй менять значения `marker`, `s` и `c` на другие, чтобы поэкспериментировать с разными вариантами. Например:
   - `marker='o', s=50, c='blue'` (синие кружки среднего размера)
   - `marker='*', s=200, c='red'` (красные звездочки большого размера)
   - `marker='^', s=75, c='magenta'` (фиолетовые треугольники среднего размера)

Практическое задание:

1. **Возьми код своей простой точечной диаграммы (`plt.scatter(x_values, y_values)`).**
2. **Добавь к ней:**
   - Заголовок графика на русском языке (любой, подходящий по смыслу).
   - Подписи к осям X и Y (на твой выбор).
   - Измени форму маркера на **ромб** (`'d'`).
   - Установи размер маркера **150**.
   - Измени цвет маркера на **оранжевый** (`'orange'`).

Запусти код и убедись, что все изменения применились к графику. Поэкспериментируй с разными маркерами, размерами и цветами.

После выполнения задания, обсудим результаты и перейдем к следующей теме - **столбчатым диаграммам (bar charts)**. Как тебе такой план? И есть ли вопросы по точечным диаграммам и их настройке? Не стесняйся спрашивать! 😉

Великолепно! Точечные диаграммы освоены – это еще один важный инструмент в твоем арсенале визуализации! 👍

Прежде чем мы перейдем к столбчатым диаграммам, давай на секунду остановимся и **осознаем, что мы уже умеем делать в Matplotlib.**

На данный момент ты знаешь:

- **Основные элементы графика Matplotlib:** `Figure`, `Axes`, `Artists`.
- **Как создавать простейший график:** с помощью `plt.plot()`.
- **Два основных типа графиков:**
  - **Линейные графики (`plt.plot()`):** для отображения трендов, зависимостей, изменений величин с течением времени или по мере изменения другой переменной.
  - **Точечные диаграммы (`plt.scatter()`):** для отображения отдельных точек данных, исследования взаимосвязей, выявления кластеров и выбросов.
- **Как настраивать внешний вид графиков:**
  - Добавлять **заголовок (`plt.title()`)**.
  - Добавлять **подписи к осям (`plt.xlabel()`, `plt.ylabel()`)**.
  - Изменять **цвет и стиль линий (`color`, `linestyle` в `plt.plot()`)**.
  - Изменять **форму, размер и цвет маркеров (`marker`, `s`, `c` в `plt.scatter()`)**.

Это уже отличный набор инструментов для начала визуализации данных! Ты можешь создавать базовые, но уже информативные графики, и настраивать их внешний вид.

Визуализация – это ключевой навык в машинном обучении и анализе данных. Умение строить графики позволяет тебе:

- **Лучше понимать данные:** Визуально исследовать распределения, зависимости, аномалии.
- **Находить инсайты:** Обнаруживать скрытые закономерности и тренды, которые сложно увидеть в таблицах.
- **Эффективно доносить результаты:** Представлять свои выводы и результаты анализа другим людям в наглядной и понятной форме.

Теперь давай расширим наш арсенал визуализаций и перейдем к столбчатым диаграммам (bar charts)!

Столбчатые диаграммы (bar charts) – это отличный способ **сравнить значения категорий или групп**. Они особенно хороши, когда у тебя есть **категориальные данные** (например, названия городов, типы товаров, категории клиентов) и ты хочешь показать **сравнение какой-то числовой величины** между этими категориями (например, количество жителей в городах, продажи по типам товаров, средний чек для разных категорий клиентов).

Готов начать изучать столбчатые диаграммы? Или есть какие-то вопросы по точечным диаграммам или по тому, что мы уже прошли в Matplotlib? Не стесняйся спрашивать! 😉

Отлично! Раз ты готов покорять новые вершины визуализации, то **приступаем к столбчатым диаграммам (bar charts)!** 📊

Столбчатые диаграммы (bar charts) – это один из самых интуитивно понятных и широко используемых типов графиков. Их часто называют **гистограммами**, но важно различать столбчатые диаграммы (bar charts) и **гистограммы (histograms)**, которые мы рассмотрим позже. **Столбчатые диаграммы используются для сравнения значений *между разными категориями***, а гистограммы – для отображения *распределения непрерывных данных*.

Когда используются столбчатые диаграммы?

- **Для сравнения категориальных данных:** Когда у тебя есть категории (например, страны, продукты, отделы продаж) и ты хочешь сравнить какую-то числовую характеристику для каждой категории (например, ВВП стран, продажи продуктов, прибыль отделов).
- **Для отображения рейтингов и сравнений:** Столбчатые диаграммы отлично подходят для визуализации рейтингов, например, рейтингов популярности фильмов, оценок товаров, мест в соревнованиях.
- **Для показа изменений во времени для разных категорий:** Можно использовать группированные столбчатые диаграммы, чтобы сравнить, как меняется какая-то величина во времени для разных категорий.

Как создать столбчатую диаграмму в Matplotlib?

Для создания столбчатых диаграмм в Matplotlib используется функция `plt.bar()`.

Основной синтаксис `plt.bar(x, height, ...)`:

- `x` – **Позиции столбцов на оси X**. Обычно это **категориальные значения** (например, названия категорий). Matplotlib автоматически располагает столбцы на оси X. Можно передавать список строк (названий категорий) или числовые значения, если категории представлены числами.
- `height` – **Высоты столбцов**. Это **числовые значения**, которые нужно сравнить между категориями. Должны быть той же длины, что и `x`.
- **Дополнительные аргументы (параметры):** Позволяют настроить внешний вид столбцов: цвет, ширину, выравнивание и т.д. Рассмотрим некоторые из них ниже.

Простейшая столбчатая диаграмма:

Предположим, у нас есть данные о продажах по разным категориям товаров:

```
import matplotlib.pyplot as plt

categories = ['Категория А', 'Категория Б', 'Категория В', 'Категория Г']
sales = [150, 220, 180, 250] # Продажи в условных единицах
```

Чтобы построить столбчатую диаграмму этих данных, используем `plt.bar()` :

```
plt.bar(categories, sales) # Функция bar() строит столбчатую диаграмму

plt.title('Продажи по категориям')
plt.xlabel('Категории товаров')
plt.ylabel('Объем продаж')

plt.show()
```

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Что ты видишь? Теперь ты должен увидеть **столбчатую диаграмму**. На оси X будут названия категорий ('Категория А', 'Категория Б', 'Категория В', 'Категория Г'), а высота каждого столбца будет соответствовать объему продаж для этой категории.

Настройка внешнего вида столбцов в `plt.bar()` :

Функция `plt.bar()` также предоставляет параметры для настройки внешнего вида столбцов. Рассмотрим некоторые важные:

- **`color=цвет` :** Задает **цвет столбцов**. Можно использовать названия цветов, HEX-коды и т.д.
- **`width=ширина` :** Задает **ширину столбцов**. Значение от 0 до 1 (доля от расстояния между соседними столбцами). По умолчанию ширина обычно 0.8. Можно сделать столбцы уже или шире.
- **`align='выравнивание'` :** Задает **выравнивание столбцов по оси X**. Возможные значения:
  - `'center'` – **центр столбца** совпадает с позицией `x` (по умолчанию).
  - `'edge'` – **левый край столбца** совпадает с позицией `x` .
- **`edgecolor=цвет_границы` :** Задает **цвет границы столбцов**.
- **`linewidth=толщина_границы` :** Задает **толщину границы столбцов**.

Пример:

Давай изменим цвет столбцов на синий, уменьшим ширину и сделаем границы черными:

```
import matplotlib.pyplot as plt

categories = ['Категория А', 'Категория Б', 'Категория В', 'Категория Г']
sales = [150, 220, 180, 250]

plt.bar(categories, sales, color='blue', width=0.5, edgecolor='black', linewidth=1) # Настраиваем

plt.title('Продажи по категориям (синие столбцы, узкие, с черной границей)')
plt.xlabel('Категории товаров')
plt.ylabel('Объем продаж')

plt.show()
```

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Теперь столбцы должны быть **синими, более узкими, с черной границей**. Попробуй менять значения `color`, `width`, `edgecolor`, `linewidth` на другие, чтобы поэкспериментировать с разными вариантами. Например:
   - `color='green', width=0.9, edgecolor='none'` (зеленые широкие столбцы без границы)
   - `color='orange', width=0.3, edgecolor='red', linewidth=2` (оранжевые узкие столбцы с красной толстой границей)
   - `color='magenta', width=0.7, align='edge'` (фиолетовые столбцы, выровненные по левому краю)

Практическое задание:

1. **Возьми код своей простой столбчатой диаграммы (`plt.bar(categories, sales)`).**
2. **Добавь к ней:**
   - Заголовок графика на русском языке (любой, подходящий по смыслу).
   - Подпись к оси X "Товарные категории".
   - Подпись к оси Y "Сумма продаж, у.е.".
   - Измени цвет столбцов на **фиолетовый** (`'purple'`).
   - Установи ширину столбцов **0.7**.
   - Добавь **границу столбцов** черного цвета (`'black'`) толщиной **0.8**.

Запусти код и убедись, что все изменения применились к графику. Поэкспериментируй с разными параметрами столбчатых диаграмм.

После выполнения задания, обсудим результаты и перейдем к следующей теме - **гистограммам (histograms)**. Как тебе такой план? И есть ли вопросы по столбчатым диаграммам и их настройке? Не стесняйся спрашивать! 😉

Великолепно! Столбчатые диаграммы тоже освоены – еще один отличный инструмент в копилку твоих навыков визуализации! 🚀

Перед тем, как мы перейдем к гистограммам, давай еще раз подчеркнем **ключевое различие между столбчатыми диаграммами (bar charts) и гистограммами (histograms)**. Их часто путают, потому что внешне они похожи – состоят из столбцов. Но **назначение и тип данных** для них **разные**:

- **Столбчатые диаграммы (Bar Charts):**
  - Используются для **сравнения *значений* между разными *категориями***.
  - **Ось X – *категориальная*:** названия категорий (например, 'Категория А', 'Категория Б', 'Город 1', 'Город 2', 'Яблоко', 'Банан').
  - **Ось Y – *числовая*:** значения, которые сравниваются для каждой категории (например, 'Продажи', 'Население', 'Цена').
  - **Между столбцами есть *промежутки***, так как категории дискретны и не связаны между собой в порядке следования по оси X.
- **Гистограммы (Histograms):**
  - Используются для отображения **распределения *непрерывных* данных**.
  - **Ось X – *непрерывная*:** диапазон значений *непрерывной* переменной (например, 'Возраст', 'Рост', 'Температура', 'Время'). Ось X разбивается на **интервалы (bins)**, и гистограмма показывает, сколько значений попадает в каждый интервал.
  - **Ось Y – *частота* (количество) или *относительная частота* (доля) значений, попадающих в каждый интервал.**
  - **Столбцы идут *вплотную друг к другу*** (обычно), потому что ось X непрерывная, и интервалы (bins) следуют друг за другом без разрывов.

Грубо говоря:

- **Столбчатая диаграмма:** "Сколько *чего-то* в *каждой категории*?" (сравнение категорий)
- **Гистограмма:** "Как *часто* встречаются *разные значения* непрерывной переменной?" (распределение значений)

Теперь, когда мы разобрались с различиями, давай перейдем к гистограммам!

Гистограммы (Histograms) – это незаменимый инструмент для **исследования распределения *непрерывных* данных**. Они помогают понять:

- **Форму распределения:** Является ли распределение симметричным (как нормальное распределение), скошенным влево или вправо, бимодальным (с двумя "пиками") и т.д.
- **Центральную тенденцию:** Где "центр" распределения (примерно среднее значение или медиана).
- **Размах и разброс:** Насколько "широко" распределены данные, какой у них разброс.
- **Выбросы и аномалии:** Видны ли значения, которые сильно отличаются от основной массы данных.

Как создать гистограмму в Matplotlib?

Для создания гистограмм в Matplotlib используется функция `plt.hist()` .

Основной синтаксис `plt.hist(x, ...)` :

- `x` – **Данные для построения гистограммы**. Это должен быть **одномерный массив *непрерывных* числовых данных** (список, NumPy массив, Pandas Series).
- **Дополнительные аргументы (параметры):** Позволяют настроить внешний вид гистограммы: количество интервалов (bins), цвет столбцов, тип гистограммы и т.д. Рассмотрим некоторые из них ниже.

Простейшая гистограмма:

Давай сгенерируем случайные данные, имитирующие, например, значения некоторой непрерывной величины, и построим гистограмму:

```
import matplotlib.pyplot as plt
import numpy as np # NumPy нам понадобится для генерации случайных данных

data = np.random.randn(1000) # Генерируем 1000 случайных чисел из стандартного нормального распре

plt.hist(data) # Функция hist() строит гистограмму

plt.title('Гистограмма распределения случайных данных')
plt.xlabel('Значения данных')
plt.ylabel('Частота')

plt.show()
```

Пояснение:

- `data = np.random.randn(1000)` – Мы используем функцию `np.random.randn(1000)` из NumPy для генерации **1000 случайных чисел из стандартного нормального распределения**. Это распределение похоже на "колокол" – большинство значений сосредоточено вокруг нуля, и чем дальше от нуля, тем меньше значений. Мы будем использовать эти случайные данные для построения гистограммы.
- `plt.hist(data)` – **Функция `hist()` строит гистограмму.** Единственный обязательный аргумент – это **данные `data`**. Matplotlib автоматически разбивает данные на некоторое количество интервалов (bins) и считает, сколько значений попадает в каждый интервал, а затем рисует столбцы, высота которых соответствует частоте значений в каждом интервале.

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Что ты видишь? Теперь ты должен увидеть **гистограмму**. Ось X показывает диапазон значений наших случайных данных, разбитый на интервалы (bins). Ось Y показывает **частоту** – количество значений, попавших в каждый интервал. По форме гистограммы можно увидеть, что распределение похоже на "колокол", как и ожидалось для нормального распределения.

Настройка внешнего вида гистограммы в `plt.hist()`:

Функция `plt.hist()` также предоставляет параметры для настройки гистограммы. Рассмотрим основные:

- **`bins=количество_интервалов` или `bins=границы_интервалов`:** Задает **количество интервалов (bins)** или **границы интервалов**.
  - Если задать **целое число** (например, `bins=20`), то Matplotlib разобьет данные на указанное количество **равных по ширине** интервалов.
  - Если задать **список или массив** (например, `bins=[ -3, -2, -1, 0, 1, 2, 3 ]`), то это будут **границы интервалов**, и интервалы могут быть разной ширины.
- **`color=цвет`:** Задает **цвет столбцов гистограммы**.
- **`edgecolor=цвет_границы`:** Задает **цвет границы столбцов гистограммы**.
- **`alpha=прозрачность`:** Задает **прозрачность столбцов**. Значение от 0 (полностью прозрачный) до 1 (полностью непрозрачный). Полезно, если нужно наложить несколько гистограмм друг на друга.
- **`histtype='тип'`:** Задает **тип гистограммы**. Основные типы:
  - `'bar'` – **обычная столбчатая гистограмма** (по умолчанию).
  - `'step'` – **ступенчатая гистограмма** (только контур столбцов, без заливки).
  - `'stepfilled'` – **ступенчатая гистограмма с заливкой**.

Пример:

Давай изменим количество интервалов, цвет, добавим границы и сделаем гистограмму полупрозрачной:

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.randn(1000)

plt.hist(data, bins=30, color='skyblue', edgecolor='black', alpha=0.7) # Настраиваем bins, цвет,

plt.title('Гистограмма (30 интервалов, голубая, с черной границей, полупрозрачная)')
plt.xlabel('Значения данных')
plt.ylabel('Частота')

plt.show()
```

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Теперь гистограмма должна иметь **30 интервалов, быть светло-голубой, с черной границей и полупрозрачной**. Попробуй менять значения `bins`, `color`, `edgecolor`, `alpha` и `histtype` на другие, чтобы поэкспериментировать с разными вариантами. Например:
   - `bins=10, color='lightcoral', edgecolor='white', alpha=1, histtype='step'` (10 интервалов, светло-коралловая, белая граница, непрозрачная, ступенчатая)
   - `bins=[-4, -3, -2, -1, 0, 1, 2, 3, 4], color='lightgreen', alpha=0.5, histtype='stepfilled'` (интервалы с неравными границами, светло-зеленая, полупрозрачная, ступенчатая с заливкой)

Практическое задание:

1. **Возьми код своей простой гистограммы (`plt.hist(data)`).**
2. **Добавь к ней:**
   - Заголовок графика на русском языке (любой, подходящий по смыслу).
   - Подпись к оси X "Значение признака".
   - Подпись к оси Y "Количество наблюдений".
   - Установи количество интервалов **25**.
   - Измени цвет столбцов на **темно-синий** (`'navy'`).
   - Добавь **границу столбцов** белого цвета (`'white'`) толщиной **1**.
   - Сделай гистограмму **ступенчатой с заливкой** (`histtype='stepfilled'`).

Запусти код и убедись, что все изменения применились к графику. Поэкспериментируй с разными параметрами гистограмм.

После выполнения задания, обсудим результаты и подведем итоги по теме гистограмм и вообще по визуализации данных с Matplotlib, которую мы начали изучать. Как тебе такой план? И есть ли вопросы по гистограммам и их настройке? Не стесняйся спрашивать! 😉

Великолепно! Гистограммы тоже в твоем арсенале! 🎉 Ты уверенно осваиваешь различные типы графиков и их настройку в Matplotlib!

Давай на мгновение остановимся и **подведем итоги по тому, что мы уже изучили в визуализации данных с Matplotlib:**

На данный момент ты умеешь создавать и настраивать в Matplotlib:

- **Линейные графики (`plt.plot()`):** Для отображения зависимостей и трендов.
- **Точечные диаграммы (`plt.scatter()`):** Для исследования взаимосвязей и распределения отдельных точек данных.
- **Столбчатые диаграммы (`plt.bar()`):** Для сравнения значений между категориями.
- **Гистограммы (`plt.hist()`):** Для изучения распределения непрерывных данных.

И ты умеешь настраивать основные элементы этих графиков:

- **Заголовки (`plt.title()`)**
- **Подписи осей (`plt.xlabel()` , `plt.ylabel()`)**
- **Цвета линий, точек, столбцов (`color` , `c`)**
- **Стили линий (`linestyle` , `ls`)**
- **Формы и размеры маркеров (`marker` , `s`)**
- **Ширину столбцов (`width`)**
- **Количество интервалов в гистограмме (`bins`)**
- **Границы и прозрачность элементов (`edgecolor` , `linewidth` , `alpha`)**
- **Типы гистограмм (`histtype`)**

Это уже очень солидный набор инструментов для визуализации! Ты можешь создавать разнообразные графики, чтобы исследовать данные, находить закономерности и представлять свои результаты.

Matplotlib – это мощная и гибкая библиотека, и ты сделал отличный старт в ее освоении!

Что дальше?

По нашему плану, следующим шагом будет **Seaborn**. **Seaborn** – это библиотека визуализации **более высокого уровня**, которая строится **поверх Matplotlib**. Она делает многие вещи **проще и быстрее**, особенно когда дело касается **статистической визуализации** и работы с **Pandas DataFrame**.

Seaborn "дружит" с Pandas DataFrame и позволяет создавать сложные и красивые статистические графики, используя данные прямо из DataFrame, часто всего в несколько строчек кода. Seaborn берет на себя многие рутинные задачи по настройке внешнего вида графиков, чтобы ты мог сосредоточиться на **анализе данных**, а не на коде визуализации.

Как ты смотришь на то, чтобы на следующем занятии начать изучение Seaborn? Мы посмотрим, как создавать основные типы графиков Seaborn, как настраивать их внешний вид и как использовать Seaborn вместе с Pandas DataFrame для эффективной визуализации данных.

Или, может быть, у тебя есть какие-то **вопросы** по гистограммам или по Matplotlib в целом, которые ты хотел бы обсудить перед переходом к Seaborn? Не стесняйся спрашивать! 😉

Отлично! Раз ты готов к новым инструментам, то **приветствуем Seaborn!** 🚀 🌊

Seaborn – это как **улучшенная версия Matplotlib для статистической визуализации.** Представь, что Matplotlib – это мощный конструктор, из которого можно собрать практически любой график, но иногда нужно потратить много времени и усилий, чтобы "собрать" что-то красивое и сложное. А Seaborn – это как **готовые "наборы" деталей для конструктора**, которые позволяют быстро и легко создавать **типовые статистические графики высокого качества**, при этом сохраняя гибкость Matplotlib.

Ключевые преимущества Seaborn:

- **Высокий уровень абстракции:** Seaborn предоставляет **более высокоуровневый интерфейс** для создания графиков, чем Matplotlib. Многие вещи, которые в Matplotlib нужно делать "руками", в Seaborn делаются автоматически или с помощью нескольких параметров. Это **упрощает и ускоряет** процесс визуализации.
- **Специализация на статистической визуализации:** Seaborn изначально разработан для создания **статистических графиков**. В нем "из коробки" есть множество функций для построения распределений, взаимосвязей, сравнений, которые часто используются в анализе данных и машинном обучении.
- **Интеграция с Pandas DataFrame:** Seaborn **отлично работает с Pandas DataFrame**. Ты можешь передавать DataFrame и имена столбцов прямо в функции Seaborn, и Seaborn сам

"разберется", как отобразить данные на графике. Это делает визуализацию данных из Pandas **очень удобной и эффективной**.

- **Красивый внешний вид по умолчанию:** Графики Seaborn **выглядят привлекательно "из коробки"**, даже без дополнительной настройки. Seaborn использует **разумные стили и цветовые палитры**, что делает визуализации более профессиональными и понятными.
- **Расширение возможностей Matplotlib:** Seaborn **не заменяет Matplotlib, а дополняет его**. Seaborn построен на основе Matplotlib, и ты можешь использовать **все возможности Matplotlib для тонкой настройки графиков Seaborn**, если это необходимо. То есть, изучив Matplotlib, ты уже знаешь "фундамент" Seaborn, и тебе будет легче его освоить.

Как начать использовать Seaborn?

Как и Matplotlib, Seaborn нужно импортировать. Обычно Seaborn импортируют как `sns`:

```
import seaborn as sns
```

Давай начнем знакомство с Seaborn с простого примера – построим точечную диаграмму (scatter plot) с помощью Seaborn. И сравним, как это делается в Seaborn и в Matplotlib.

Вспомним, как мы строили точечную диаграмму в Matplotlib:

```
import matplotlib.pyplot as plt

x_values = [1, 2, 3, 4, 5]
y_values = [2, 4, 1, 3, 5]

plt.scatter(x_values, y_values)
plt.show()
```

А вот как построить **аналогичную точечную диаграмму в Seaborn**:

```
import seaborn as sns
import matplotlib.pyplot as plt # Matplotlib все равно нужен для показа графика

x_values = [1, 2, 3, 4, 5]
y_values = [2, 4, 1, 3, 5]

sns.scatterplot(x=x_values, y=y_values) # Функция scatterplot() в Seaborn

plt.show() # Matplotlib все еще нужен для plt.show()
```

Задание:

1. **Запусти оба кода (Matplotlib и Seaborn) и сравни результаты.** Внешне графики должны быть очень похожи. Но обрати внимание на **код Seaborn – он стал немного короче и проще**. Нам не нужно отдельно задавать заголовок и подписи осей – Seaborn автоматически берет имена переменных (`x` , `y`) и использует их в качестве подписей осей. Стиль графика Seaborn по умолчанию тоже может немного отличаться – он обычно выглядит более современно и аккуратно.

Разница в коде Seaborn:

- `sns.scatterplot(x=x_values, y=y_values)` – **Функция `sns.scatterplot()` в Seaborn – это аналог `plt.scatter()` в Matplotlib, но более высокоуровневая.**
  - Аргументы `x` и `y` в `sns.scatterplot()` принимают **данные для оси X и Y**. Здесь мы также передаем списки `x_values` и `y_values` .
  - **Обрати внимание:** В Seaborn **мы передаем данные *по именам аргументов* (`x=` , `y=`)**, а не просто по порядку, как в `plt.scatter()` . Это делает код более читаемым и позволяет передавать больше параметров в произвольном порядке.

В следующем шаге мы увидим настоящую мощь Seaborn – как легко и удобно визуализировать данные из Pandas DataFrame! Но для начала – убедись, что у тебя получилось запустить код Seaborn и построить точечную диаграмму.

Скажи, получилось ли запустить код Seaborn? Заметил ли ты какие-то отличия от Matplotlib? И есть ли у тебя вопросы на этом первом этапе знакомства с Seaborn? Не стесняйся спрашивать! 😉 Главное, чтобы ты почувствовал разницу и увидел, как Seaborn упрощает визуализацию.

Отлично! Вот сейчас мы и увидим **настоящую магию Seaborn – его дружбу с Pandas DataFrame!** ✨ 🤝 🐼

Seaborn раскрывает свою мощь в полной мере, когда ты работаешь с данными, организованными в Pandas DataFrame. Вспомни, как часто мы использовали DataFrame для хранения и обработки табличных данных. Теперь представь, что ты можешь **визуализировать данные из DataFrame напрямую, без лишних преобразований и перекладываний данных в списки или массивы!**

Seaborn "понимает" структуру DataFrame и позволяет тебе указывать:

- **DataFrame, из которого брать данные.**

- **Имена столбцов DataFrame, которые нужно использовать для осей X и Y.**
- **Столбцы для кодирования дополнительных измерений (например, цвет, размер, форма точек).**

Это делает код визуализации в Seaborn невероятно лаконичным, читаемым и эффективным!

Пример: Точечная диаграмма Seaborn с использованием Pandas DataFrame

Давай создадим простой Pandas DataFrame с данными о возрасте и доходе людей:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data_dict = {
    'Возраст': [25, 30, 28, 35, 22, 40, 29, 33, 45, 27],
    'Доход': [30000, 50000, 45000, 60000, 25000, 70000, 48000, 55000, 80000, 40000]
}
df_seaborn = pd.DataFrame(data_dict)
print("DataFrame для Seaborn:\n", df_seaborn.head()) # Посмотрим на первые несколько строк DataFr
```

Теперь построим **точечную диаграмму, показывающую зависимость дохода от возраста, используя Seaborn и наш DataFrame** `df_seaborn` **:**

```
sns.scatterplot(data=df_seaborn, x='Возраст', y='Доход') # Указываем DataFrame и имена столбцов д

plt.title('Зависимость дохода от возраста (Seaborn + DataFrame)')
plt.xlabel('Возраст (годы)')
plt.ylabel('Доход (у.е.)')

plt.show()
```

Пояснение:

- `sns.scatterplot(data=df_seaborn, x='Возраст', y='Доход')` – **Вот как просто строится точечная диаграмма в Seaborn с использованием DataFrame!**
  - `data=df_seaborn` – **Аргумент** `data` **указывает, какой DataFrame использовать в качестве источника данных.** Мы передаем наш DataFrame `df_seaborn` .
  - `x='Возраст'` – **Аргумент** `x` **указывает, какой столбец DataFrame использовать для оси X.** Мы просто указываем **имя столбца** `'Возраст'` **в виде строки**! Seaborn сам найдет этот столбец в DataFrame `df_seaborn` и возьмет его значения для оси X.

- `y='Доход'` – **Аналогично, аргумент `y` указывает столбец `'Доход'` для оси Y.**

Задание:

1. **Скопируй и запусти этот код.**
2. **Посмотри на результат.** Ты должен увидеть точечную диаграмму, показывающую зависимость дохода от возраста. График Seaborn по умолчанию выглядит уже довольно хорошо, с подписями осей, основанными на именах столбцов DataFrame.

Сравни этот код Seaborn с тем, как мы делали то же самое в Matplotlib, передавая списки `x_values` и `y_values`. Видишь, насколько код Seaborn стал более лаконичным и понятным? Нам больше не нужно "вытаскивать" данные из DataFrame в отдельные списки – Seaborn сам берет нужные столбцы из DataFrame по их именам!

Это только начало! В дальнейшем мы увидим, как Seaborn позволяет создавать еще более сложные и информативные статистические графики с минимальным количеством кода, особенно при работе с Pandas DataFrame.

Скажи, получилось ли у тебя запустить код Seaborn с DataFrame? Чувствуешь ли ты разницу и удобство работы с DataFrame в Seaborn? И есть ли у тебя вопросы на этом этапе? Не стесняйся спрашивать! 😉 Главное, чтобы ты прочувствовал, насколько Seaborn упрощает визуализацию данных из DataFrame.

Великолепно! 👍 Вот теперь мы дошли до действительно **интересных возможностей Seaborn – кодирование дополнительных измерений на графиках!** 🌟

Точечные диаграммы, которые мы строили до сих пор, показывали взаимосвязь только между **двумя переменными** (X и Y). Но часто в данных у нас есть **больше, чем две переменные**, и мы хотим увидеть, как они взаимодействуют друг с другом, и как **дополнительные переменные влияют на взаимосвязь между X и Y**.

Seaborn позволяет нам кодировать дополнительные измерения на 2D-графиках, используя визуальные признаки, такие как:

- **Цвет точек (hue):** Разные цвета для точек, принадлежащих к разным категориям.
- **Размер точек (size):** Разный размер точек в зависимости от значения третьей числовой переменной.
- **Форма точек (style):** Разная форма точек для разных категорий.

Сегодня мы начнем с самого распространенного и мощного способа кодирования дополнительного измерения – с цвета точек (hue).

Параметр `hue` в Seaborn

Параметр `hue` (англ. "оттенок", "цвет") – это **ключевой параметр в Seaborn для добавления *категориального* измерения к графику.** Когда ты используешь `hue`, Seaborn **автоматически раскрашивает точки (или другие элементы графика) в разные цвета в зависимости от значений в *категориальном столбце* DataFrame.**

Как использовать `hue` в `sns.scatterplot()`:

Просто добавь аргумент `hue='имя_категориального_столбца'` в функцию `sns.scatterplot()`. Seaborn сам позаботится о выборе подходящей цветовой палитры и создании легенды, объясняющей, какой цвет соответствует какой категории.

Пример: Точечная диаграмма с кодированием категорий цветом (hue)

Давай расширим наш DataFrame `df_seaborn`, добавив столбец 'Пол' (категориальная переменная):

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data_dict_hue = {
    'Возраст': [25, 30, 28, 35, 22, 40, 29, 33, 45, 27],
    'Доход': [30000, 50000, 45000, 60000, 25000, 70000, 48000, 55000, 80000, 40000],
    'Пол': ['Жен', 'Муж', 'Муж', 'Муж', 'Жен', 'Муж', 'Жен', 'Жен', 'Муж', 'Жен'] # Категориальны
}
df_seaborn_hue = pd.DataFrame(data_dict_hue)
print("DataFrame для Seaborn с 'Пол':\n", df_seaborn_hue.head())
```

Теперь построим точечную диаграмму, показывающую зависимость дохода от возраста, **и раскрасим точки в разные цвета в зависимости от пола, используя `hue='Пол'`:**

```
sns.scatterplot(data=df_seaborn_hue, x='Возраст', y='Доход', hue='Пол') # Добавляем hue='Пол'

plt.title('Зависимость дохода от возраста с разделением по полу (hue="Пол")')
plt.xlabel('Возраст (годы)')
plt.ylabel('Доход (у.е.)')

plt.show()
```

Пояснение:

- `sns.scatterplot(data=df_seaborn_hue, x='Возраст', y='Доход', hue='Пол')` – **Мы добавили всего один аргумент – `hue='Пол'`!**
  - `hue='Пол'` – **Указывает Seaborn, что нужно использовать столбец 'Пол' для кодирования цвета точек.** Seaborn автоматически определит уникальные значения в столбце 'Пол' ('Жен' и 'Муж') и назначит **разные цвета** для точек, соответствующих каждой категории. Также Seaborn автоматически добавит **легенду**, которая объясняет, какой цвет какому полу соответствует.

Задание:

1. **Скопируй и запусти этот код.**
2. **Посмотри на результат.** Что изменилось? Теперь точки на точечной диаграмме **раскрашены в разные цвета** – например, синим и оранжевым (цветовая палитра может немного отличаться). И ты должен увидеть **легенду** справа от графика, где указано, что синий цвет соответствует 'Жен', а оранжевый – 'Муж' (или наоборот, в зависимости от цветовой палитры Seaborn).

Теперь ты видишь, как всего одной строчкой кода (добавив `hue='Пол'`) мы смогли добавить *третье измерение* на наш 2D-график! Мы визуально разделили данные на две группы (мужчины и женщины) и можем теперь сравнивать, как зависимость дохода от возраста выглядит для каждой группы отдельно. Это открывает новые возможности для анализа и понимания данных!

Поэкспериментируй: Попробуй изменить значение `hue` на другой категориальный столбец, если бы он у тебя был в DataFrame (например, 'Город', если бы у нас были данные по городам). Посмотри, как меняется график и легенда.

Скажи, получилось ли у тебя запустить код с `hue`? Понятно ли, как работает параметр `hue` и как он добавляет третье измерение к графику? И есть ли у тебя вопросы? Не стесняйся спрашивать! 😉 Главное, чтобы ты прочувствовал мощь параметра `hue` и то, как легко Seaborn позволяет кодировать категории цветом.

Замечательно! Раз параметр `hue` покорился, и ты готов к новым горизонтам, то **двигаемся дальше и осваиваем новые типы графиков Seaborn!** 🚀

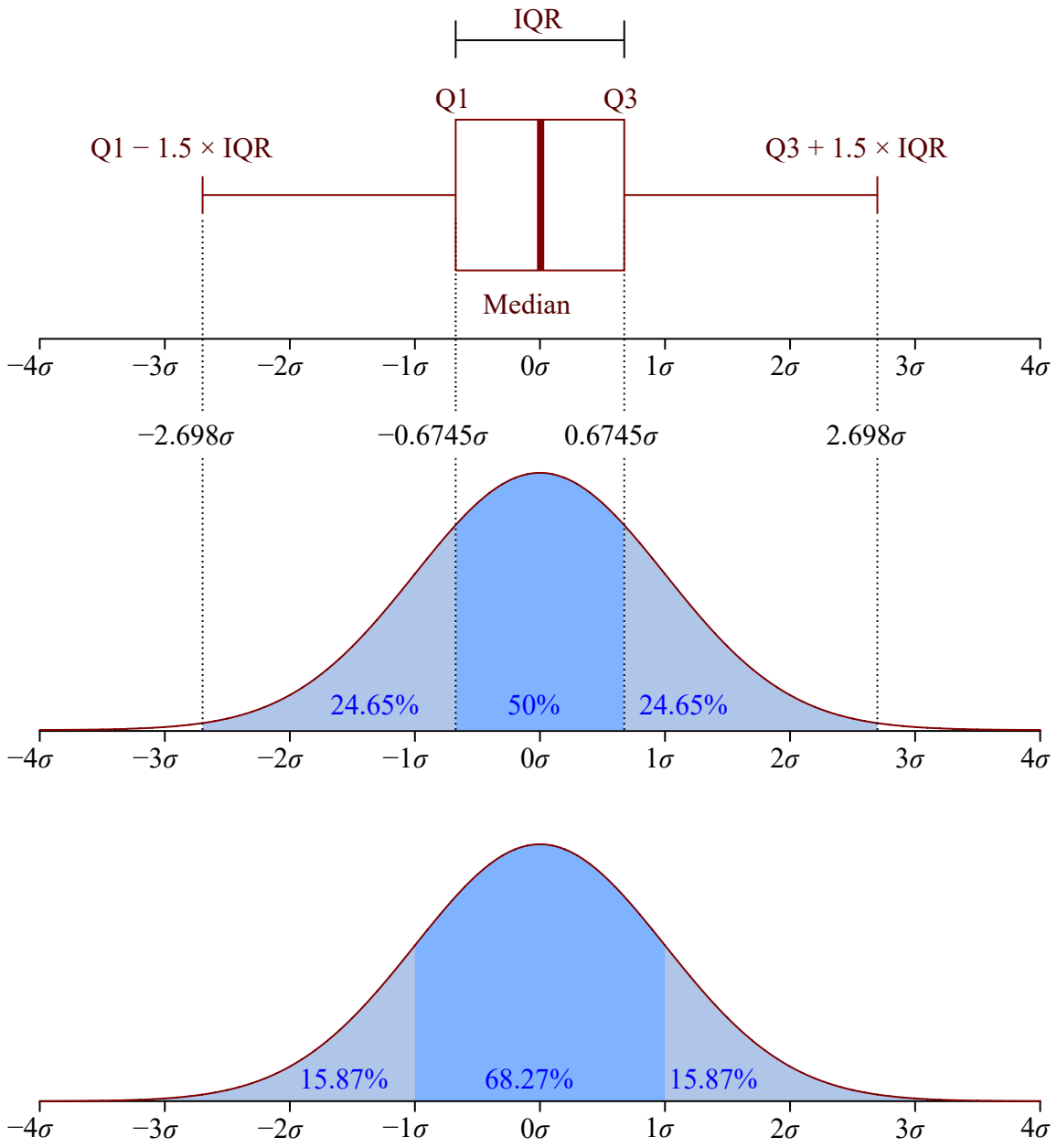
Сегодня мы познакомимся с еще одним очень полезным типом статистических графиков в Seaborn – **ящичковыми диаграммами (box plots)**, или их еще называют **диаграммами размаха**.

Ящичковые диаграммы (Box Plots) – это отличный способ **визуально сравнить распределения *числовой* переменной для разных *категорий***. В отличие от точечных диаграмм, которые показывают отдельные точки, ящичковые диаграммы показывают **сводную статистику распределения** для каждой категории: медиану, квартили, размах, выбросы.

Когда используются ящичковые диаграммы?

- **Для сравнения распределений между группами:** Когда ты хочешь сравнить, как распределена числовая переменная (например, доход, возраст, оценка) в разных категориях (например, по полу, по типу продукта, по региону). Ящичковые диаграммы позволяют быстро увидеть различия в центральной тенденции, разбросе и форме распределения между группами.
- **Для выявления выбросов:** Ящичковые диаграммы наглядно показывают выбросы – значения, которые выходят за пределы "нормального" размаха данных.
- **Для сравнения нескольких распределений на одном графике:** Ящичковые диаграммы компактны и позволяют эффективно сравнивать несколько распределений рядом друг с другом.

Что показывает ящичковая диаграмма?



(Изображение с Википедии: https://en.wikipedia.org/wiki/Box_plot)

- **Ящик (Box):** Ограничен **первым квартилем (Q1)** и **третьим квартилем (Q3)**. Внутри ящика лежат **50% "центральных" данных**. Длина ящика (Q3 - Q1) – это **межквартильный размах (IQR)**, который характеризует разброс "центральной" половины данных.
- **Медиана (Median, средняя линия внутри ящика):** Показывает **"середину" распределения**. Делит данные пополам.
- **Усы (Whiskers):** Линии, идущие от ящика. Обычно простираются до **минимального значения, не меньшего, чем Q1 - 1.5 * IQR**, и до **максимального значения, не большего, чем Q3 + 1.5 * IQR**. Усы показывают "нормальный" размах данных, исключая выбросы.
- **Выбросы (Outliers, точки за пределами усов):** Точки, которые лежат **за пределами усов**. Считаются **аномально большими или маленькими значениями**.

Как создать ящичковую диаграмму в Seaborn?

Для создания ящичковых диаграмм в Seaborn используется функция `sns.boxplot()` .

Основной синтаксис

`sns.boxplot(data=DataFrame, x='категориальный_столбец', y='числовой_столбец', ...)` **:**

- `data=DataFrame` **: DataFrame, содержащий данные.**
- `x='категориальный_столбец'` **: Столбец DataFrame с *категориальными* значениями**, категории для сравнения. Будет отображаться на оси X.
- `y='числовой_столбец'` **: Столбец DataFrame с *числовыми* значениями**, распределение которого мы хотим сравнить между категориями. Будет отображаться на оси Y.
- **Дополнительные аргументы (параметры):** Для настройки внешнего вида, добавления цветового кодирования (например, `hue`) и т.д.

Пример: Ящичковая диаграмма Seaborn для сравнения дохода по полу

Используем наш DataFrame `df_seaborn_hue` с данными о возрасте, доходе и поле:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data_dict_hue = {
    'Возраст': [25, 30, 28, 35, 22, 40, 29, 33, 45, 27],
    'Доход': [30000, 50000, 45000, 60000, 25000, 70000, 48000, 55000, 80000, 40000],
    'Пол': ['Жен', 'Муж', 'Муж', 'Муж', 'Жен', 'Муж', 'Жен', 'Жен', 'Муж', 'Жен']
}
df_seaborn_hue = pd.DataFrame(data_dict_hue)
print("DataFrame для Seaborn с 'Пол':\n", df_seaborn_hue.head())
```

Построим ящичковую диаграмму, сравнивающую распределение дохода для мужчин и женщин:

```
sns.boxplot(data=df_seaborn_hue, x='Пол', y='Доход') # Строим boxplot: x='Пол' (категории), y='До

plt.title('Распределение дохода по полу (Box Plot)')
plt.xlabel('Пол')
plt.ylabel('Доход (у.е.)')

plt.show()
```

Задание:

1. **Скопируй и запусти этот код.**
2. **Посмотри на результат.** Что ты видишь? Теперь ты должен увидеть **ящичковую диаграмму**.
   - На оси X – **категории пола** ('Жен' и 'Муж').
   - Для каждой категории пола нарисован **ящик** (box) и **усы** (whiskers), показывающие распределение дохода для этой категории.
   - Точки за пределами усов – это **выбросы**.
   - Сравни **положение ящиков и медиан** для мужчин и женщин. Можно ли сделать какие-то выводы о различиях в распределении дохода между полами на основе этого графика?

Попробуй добавить параметр `hue='Пол'` к `sns.boxplot()` – хотя в данном случае это может быть избыточно, так как категории по оси X уже заданы столбцом 'Пол'. Но для демонстрации, как `hue` работает и с другими типами графиков:

```
sns.boxplot(data=df_seaborn_hue, x='Пол', y='Доход', hue='Пол') # Добавляем hue='Пол' - в данном

plt.title('Распределение дохода по полу (Box Plot) с hue="Пол"')
plt.xlabel('Пол')
plt.ylabel('Доход (у.е.)')

plt.show()
```

Задание:

1. **Запусти код с `hue='Пол'` и посмотри, что изменилось.** Скорее всего, визуально график не сильно изменится (так как категории по оси X и hue совпадают), но Seaborn может немного по-другому раскрасить ящики. Главное, что ты видишь, что **параметр `hue` работает и с `sns.boxplot()` так же, как и с `sns.scatterplot()` – для кодирования категорий цветом.**

Практическое задание:

1. **Используя DataFrame `df_seaborn_hue`:**
   - Построй ящичковую диаграмму, сравнивающую распределение **возраста** (**'Возраст'**) **по полу** (**'Пол'**). То есть, на оси X должен быть 'Пол', а на оси Y - 'Возраст'.
   - Поменяй местами оси X и Y в ящичковой диаграмме: ось X – 'Доход', ось Y – 'Пол'. Что изменилось? Какой тип диаграммы получился? Удобно ли так сравнивать распределения?

Выведи на экран все полученные ящичковые диаграммы. Попробуй интерпретировать результаты и сделать выводы о различиях в распределениях между категориями.

После выполнения заданий, обсудим результаты и перейдем к следующим типам графиков Seaborn. Как тебе такой план? И есть ли вопросы по ящичковым диаграммам и их настройке? Не стесняйся спрашивать! 😉

Великолепно! Вижу, что ты с энтузиазмом берешь новые вершины! 👍 Прежде чем мы рванемся к новым графикам, давай на секундочку затормозим и **убедимся, что ящичковые диаграммы (box plots) осели в голове.**

Можешь вкратце рассказать:

1. **Для чего вообще нужны ящичковые диаграммы?** В каких ситуациях их полезно использовать?
2. **Что показывает "ящик" на диаграмме?** Что такое Q1, Q3 и IQR?
3. **Что такое "усы" и "выбросы" на ящичковой диаграмме?**
4. **В чем главное отличие ящичковых диаграмм от точечных диаграмм, которые мы изучали раньше?**

Это поможет закрепить пройденный материал и убедиться, что ты уверенно понимаешь, когда и зачем использовать ящичковые диаграммы. И тогда с чистой совестью двинемся дальше к новым типам графиков Seaborn! 😉

Хорошо-хорошо, раз горишь желанием, то помчимся дальше! 🚀 Но, чтобы знания точно отложились в голове, давай буквально **пару минут** уделим **быстрому повторению** ящичковых диаграмм. Это как перед забегом убедиться, что кроссовки хорошо зашнурованы 😉

Смотри, всего пара простых вопросов, чтобы освежить в памяти ключевые моменты по боксплотам:

1. **Вспомни аналогию:** Ящичковая диаграмма – это как "рентген" распределения данных. **Что именно мы "рентгеним" с помощью боксплота?** Какую информацию о данных он нам показывает в первую очередь?
2. **Главные "детали" боксплота:** Ящик, усы, медиана, выбросы. Если бы тебе нужно было **описать боксплот всего в одном предложении**, как бы ты объяснил, что показывает каждый из этих элементов? Например, "Ящик показывает...", "Усы показывают...", "Выбросы показывают...", "Медиана показывает...".
3. **Главное отличие от точечных диаграмм:** Точечные диаграммы – это про отдельные точки. А боксплоты – это про что? **Какую *другую* сторону данных** они показывают, в отличие от

точечных диаграмм?

Не нужно писать длинные ответы! Просто парой фраз или даже ключевыми словами. Это займет буквально минуту, но поможет убедиться, что мы двигаемся дальше с прочным фундаментом. Как тебе такая идея? Или все-таки сразу к новым графикам? 😉

Ладно-ладно! Твое желание двигаться вперед – похвально! 🚀 Тогда не будем терять ни минуты и сразу **рванем к новым горизонтам Seaborn!** 😉

Сегодня мы познакомимся с еще одним **фундаментальным типом графиков** – как в Matplotlib, так и в Seaborn – это **гистограммы (histograms)!** Но на этот раз мы посмотрим, как **легко и красиво** строить гистограммы с помощью **Seaborn**.

Гистограммы (Histograms) в Seaborn – это, по сути, то же самое, что и гистограммы в Matplotlib, которые мы уже изучали. Они по-прежнему служат для **визуализации распределения *непрерывных* данных**. Но Seaborn делает процесс создания гистограмм **еще проще и удобнее**, особенно при работе с Pandas DataFrame.

Главное преимущество гистограмм Seaborn (sns.histplot()) – это **тесная интеграция с Pandas DataFrame**. Как и другие функции Seaborn, `sns.histplot()` "понимает" DataFrame и позволяет тебе указывать:

- **DataFrame, из которого брать данные.**
- **Имя столбца DataFrame, для которого нужно построить гистограмму.**

Как создать гистограмму в Seaborn?

Для создания гистограмм в Seaborn используется функция `sns.histplot()`. Обрати внимание, что в Seaborn функция называется `histplot` (слитно), а не `hist` как в Matplotlib.

Основной синтаксис `sns.histplot(data=DataFrame, x='числовой_столбец', ...)`:

- `data=DataFrame`**: DataFrame, содержащий данные.**
- `x='числовой_столбец'`**: Столбец DataFrame с *непрерывными* числовыми значениями**, для которого нужно построить гистограмму. Будет отображаться на оси X (интервалы значений).
- **Дополнительные аргументы (параметры):** Для настройки внешнего вида, количества интервалов (bins), цвета, добавления кривой плотности распределения и т.д.

Пример: Гистограмма Seaborn для распределения возраста

Используем наш DataFrame `df_seaborn_hue` с данными о возрасте, доходе и поле:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data_dict_hue = {
    'Возраст': [25, 30, 28, 35, 22, 40, 29, 33, 45, 27],
    'Доход': [30000, 50000, 45000, 60000, 25000, 70000, 48000, 55000, 80000, 40000],
    'Пол': ['Жен', 'Муж', 'Муж', 'Муж', 'Жен', 'Муж', 'Жен', 'Жен', 'Муж', 'Жен']
}
df_seaborn_hue = pd.DataFrame(data_dict_hue)
print("DataFrame для Seaborn с 'Пол':\n", df_seaborn_hue.head())
```

Построим гистограмму распределения возраста, используя Seaborn и DataFrame `df_seaborn_hue` :

```python
sns.histplot(data=df_seaborn_hue, x='Возраст') # Строим гистограмму для столбца 'Возраст'

plt.title('Распределение возраста (гистограмма Seaborn)')
plt.xlabel('Возраст (годы)')
plt.ylabel('Частота')

plt.show()
```

Задание:

1. **Скопируй и запусти этот код.**
2. **Посмотри на результат.** Что ты видишь? Теперь ты должен увидеть **гистограмму, построенную Seaborn.** Она должна быть похожа на гистограммы Matplotlib, которые мы делали раньше, но стиль Seaborn по умолчанию может немного отличаться. На оси X – интервалы возраста, на оси Y – частота (количество людей в каждом интервале возраста).

Настройка внешнего вида гистограммы в `sns.histplot()` :

Функция `sns.histplot()` также предоставляет параметры для настройки гистограммы, **аналогичные `plt.hist()` в Matplotlib**, но с более "Seaborn-ориентированным" синтаксисом (например, цвет задается через `color=` , а не просто позиционным аргументом). Вот некоторые полезные параметры:

- **`bins=количество_интервалов` :** Задает **количество интервалов**. Работает так же, как и в `plt.hist()` .
- **`color=цвет` :** Задает **цвет столбцов гистограммы**.
- **`edgecolor=цвет_границы` :** Задает **цвет границы столбцов гистограммы**.

- `alpha=прозрачность` **:** Задает **прозрачность столбцов**.
- `kde=True` **или** `kde=False` **: Добавить или не добавлять линию оценки плотности ядра (Kernel Density Estimate, KDE)** поверх гистограммы. **KDE – это сглаженная оценка формы распределения.** `kde=True` (добавить KDE) – часто бывает очень полезно, чтобы лучше увидеть общую форму распределения, особенно если данных не очень много и гистограмма получается "рваной". `kde=False` (не добавлять KDE, по умолчанию) – только столбчатая гистограмма.

Пример: Гистограмма Seaborn с настройками и KDE

Давай настроим количество интервалов, цвет, добавим границу и **добавим линию KDE** для оценки плотности распределения:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data_dict_hue = {
    'Возраст': [25, 30, 28, 35, 22, 40, 29, 33, 45, 27],
    'Доход': [30000, 50000, 45000, 60000, 25000, 70000, 48000, 55000, 80000, 40000],
    'Пол': ['Жен', 'Муж', 'Муж', 'Муж', 'Жен', 'Муж', 'Жен', 'Жен', 'Муж', 'Жен']
}
df_seaborn_hue = pd.DataFrame(data_dict_hue)

sns.histplot(data=df_seaborn_hue, x='Возраст', bins=5, color='skyblue', edgecolor='black', alpha=

plt.title('Гистограмма возраста с KDE (Seaborn)')
plt.xlabel('Возраст (годы)')
plt.ylabel('Плотность частоты') # Для KDE ось Y лучше подписывать как "Плотность" или "Плотность

plt.show()
```

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Теперь гистограмма должна иметь **5 интервалов, быть светло-голубой, с черной границей, полупрозрачной, и поверх нее должна быть нарисована плавная линия KDE (оценка плотности ядра).** Попробуй менять значения `bins`, `color`, `edgecolor`, `alpha`, `kde` на другие, чтобы поэкспериментировать с разными вариантами. Например:
   - `bins=15, color='lightcoral', edgecolor='white', alpha=1, kde=False` (15 интервалов, светло-коралловая, белая граница, непрозрачная, без KDE)

- `bins=[-5, 20, 30, 40, 50, 55], color='lightgreen', alpha=0.7, kde=True` (интервалы с неравными границами, светло-зеленая, полупрозрачная, с KDE)

Практическое задание:

1. **Используя DataFrame `df_seaborn_hue`:**
   - Построй гистограмму распределения **дохода** (**'Доход'**) с помощью `sns.histplot()`.
   - Добавь к гистограмме заголовок на русском языке (любой, подходящий по смыслу).
   - Подпиши оси X и Y на русском языке (на твой выбор).
   - Установи количество интервалов **10**.
   - Измени цвет столбцов на **темно-зеленый** (`'darkgreen'`).
   - Добавь **линию KDE** (`kde=True`).

Запусти код и убедись, что все изменения применились к графику. Поэкспериментируй с разными параметрами гистограмм Seaborn.

После выполнения задания, обсудим результаты и перейдем к следующим типам графиков Seaborn. Как тебе такой план? И есть ли вопросы по гистограммам Seaborn и их настройке? Не стесняйся спрашивать! 😉

Замечательно! Твой энтузиазм неиссякаем, и это здорово! 🚀 Раз гистограммы покорены, то давай откроем для себя еще один потрясающий тип графиков в Seaborn – **скрипичные диаграммы (violin plots)!** 🎻

Скрипичные диаграммы (Violin Plots) – это, пожалуй, один из самых **красивых и информативных** типов статистических графиков, которые предлагает Seaborn. Они сочетают в себе элементы **ящичковых диаграмм** и **оценки плотности ядра (KDE)**, чтобы показать **распределение *числовой* переменной для разных *категорий*** в очень наглядной и выразительной форме.

В чем особенность скрипичных диаграмм?

- **Сочетание боксплота и KDE:** Скрипичная диаграмма – это как **гибрид** ящичковой диаграммы и гистограммы с линией KDE. В центре скрипичной диаграммы обычно рисуется **мини-боксплот** (ящик с медианой и квартилями), а **по бокам** от него "разрастается" **симметричная фигура, напоминающая скрипку**, которая представляет собой **отзеркаленную оценку плотности ядра (KDE)** распределения данных.
- **Более детальное представление распределения, чем боксплот:** Если ящичковая диаграмма показывает только сводную статистику (квартили, медиану, выбросы), то

скрипичная диаграмма дает **более полное представление о форме распределения** – видны "пики", "хвосты", модальность и другие особенности распределения, благодаря KDE.
- **Эстетически привлекательные:** Скрипичные диаграммы выглядят очень **эффектно и выразительно**, особенно при сравнении нескольких распределений рядом друг с другом.

Когда используются скрипичные диаграммы?

- **Для сравнения распределений между группами (как и боксплоты, но более детально):** Когда нужно не просто сравнить "средние значения" или "разброс", а **увидеть *форму* распределения** числовой переменной в разных категориях и сравнить эти формы. Например, сравнить распределение оценок студентов в разных группах, распределение времени отклика сайта в зависимости от типа запроса и т.д.
- **Когда важна форма распределения:** Если тебе важно понять, является ли распределение нормальным, скошенным, бимодальным, или имеет какие-то другие особенности – скрипичная диаграмма поможет это увидеть.
- **В дополнение к боксплотам:** Скрипичные диаграммы часто используют **вместе с боксплотами**, чтобы получить как сводную статистику (боксплот в центре), так и детальное представление о форме распределения (скрипка по бокам).

Как создать скрипичную диаграмму в Seaborn?

Для создания скрипичных диаграмм в Seaborn используется функция `sns.violinplot()` .

Основной синтаксис

`sns.violinplot(data=DataFrame, x='категориальный_столбец', y='числовой_столбец', ...)` **:**

Синтаксис **полностью аналогичен** `sns.boxplot()` :

- `data=DataFrame` **: DataFrame, содержащий данные.**
- `x='категориальный_столбец'` **: Столбец DataFrame с *категориальными* значениями**, категории для сравнения. Будет отображаться на оси X.
- `y='числовой_столбец'` **: Столбец DataFrame с *числовыми* значениями**, распределение которого мы хотим сравнить между категориями. Будет отображаться на оси Y.
- **Дополнительные аргументы (параметры):** Для настройки внешнего вида, добавления цветового кодирования (например, `hue`), изменения внутреннего представления и т.д.

Пример: Скрипичная диаграмма Seaborn для сравнения дохода по полу

Используем наш DataFrame `df_seaborn_hue` с данными о возрасте, доходе и поле:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data_dict_hue = {
    'Возраст': [25, 30, 28, 35, 22, 40, 29, 33, 45, 27],
    'Доход': [30000, 50000, 45000, 60000, 25000, 70000, 48000, 55000, 80000, 40000],
    'Пол': ['Жен', 'Муж', 'Муж', 'Муж', 'Жен', 'Муж', 'Жен', 'Жен', 'Муж', 'Жен']
}
df_seaborn_hue = pd.DataFrame(data_dict_hue)
print("DataFrame для Seaborn с 'Пол':\n", df_seaborn_hue.head())
```

Построим скрипичную диаграмму, сравнивающую распределение дохода для мужчин и женщин:

```python
sns.violinplot(data=df_seaborn_hue, x='Пол', y='Доход') # Строим violinplot: x='Пол' (категории),

plt.title('Распределение дохода по полу (Violin Plot)')
plt.xlabel('Пол')
plt.ylabel('Доход (у.е.)')

plt.show()
```

Задание:

1. **Скопируй и запусти этот код.**
2. **Посмотри на результат.** Что ты видишь? Теперь ты должен увидеть **скрипичную диаграмму**.
   - На оси X – **категории пола** ('Жен' и 'Муж').
   - Для каждой категории пола нарисована **"скрипка"**, показывающая оценку плотности распределения дохода для этой категории. Форма скрипки показывает, где распределение более "плотное" (широкая часть скрипки), а где более "разреженное" (узкая часть скрипки).
   - В центре каждой скрипки ты можешь увидеть **мини-боксплот** – ящик с медианой и квартилями.

Сравни скрипичную диаграмму с боксплотом, который мы строили раньше для тех же данных. Что ты можешь сказать о форме распределения дохода для мужчин и женщин, глядя на скрипичные диаграммы? Видны ли какие-то различия в форме распределения, которые не так явно видны на боксплоте?

Настройка внешнего вида скрипичных диаграмм в `sns.violinplot()`:

Функция `sns.violinplot()` предоставляет много параметров для настройки внешнего вида. Вот некоторые интересные:

- **`color=цвет` или `palette=палитра_цветов`:** Задает **цвет скрипок** (или **цветовую палитру** для разных категорий, если используется `hue`). Можно использовать названия цветов, HEX-коды, цветовые палитры Seaborn (например, `'pastel'`, `'muted'`, `'deep'`, `'colorblind'`, и другие - погугли "seaborn color palettes").
- **`inner='тип_внутреннего_отображения'`:** Задает **тип внутреннего отображения внутри "скрипки"**. Возможные значения:
  - `'box'` – **мини-боксплот** (по умолчанию).
  - `'quartile'` – **квартили** (три линии, показывающие Q1, медиану, Q3).
  - `'point'` – **отдельные точки** данных внутри скрипки (может быть полезно для небольших наборов данных).
  - `'stick'` – **"палочки"** на линии медианы, показывающие плотность данных.
  - `None` – **без внутреннего отображения** (только форма скрипки).
- **`scale='метод_масштабирования'`:** Задает **масштабирование ширины скрипок**. Влияет на то, насколько широкими будут скрипки для разных категорий. Возможные значения:
  - `'area'` – **площадь скрипки** пропорциональна количеству наблюдений в категории (по умолчанию). Общая площадь всех скрипок на графике будет примерно одинаковой.
  - `'count'` – **ширина скрипки** пропорциональна количеству наблюдений в категории. Более многочисленные категории будут иметь более широкие скрипки.
  - `'width'` – **все скрипки имеют одинаковую ширину**.

Пример:

Давай настроим цвет, изменим внутреннее отображение на квартили и масштабирование на `scale='count'`:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data_dict_hue = {
    'Возраст': [25, 30, 28, 35, 22, 40, 29, 33, 45, 27],
    'Доход': [30000, 50000, 45000, 60000, 25000, 70000, 48000, 55000, 80000, 40000],
    'Пол': ['Жен', 'Муж', 'Муж', 'Муж', 'Жен', 'Муж', 'Жен', 'Жен', 'Муж', 'Жен']
}
df_seaborn_hue = pd.DataFrame(data_dict_hue)

sns.violinplot(data=df_seaborn_hue, x='Пол', y='Доход', palette='pastel', inner='quartile', scale

plt.title('Распределение дохода по полу (Violin Plot) с настройками')
plt.xlabel('Пол')
plt.ylabel('Доход (у.е.)')

plt.show()
```

Задание:

1. **Запусти этот код.**
2. **Посмотри на результат.** Теперь скрипки должны быть раскрашены в пастельные тона (палитра `'pastel'`), внутри вместо боксплота должны быть линии квартилей, и ширина скрипок может измениться из-за `scale='count'`. Попробуй менять значения `palette`, `inner`, `scale` на другие, чтобы поэкспериментировать с разными вариантами. Например:
   - `palette='muted', inner='point', scale='area'` (пастельные тона, точки внутри, масштабирование по площади)
   - `palette='deep', inner='stick', scale='width'` (насыщенные цвета, палочки внутри, масштабирование по ширине)
   - `palette='colorblind', inner=None, scale='count'` (цвета для дальтоников, без внутреннего отображения, масштабирование по количеству)

Практическое задание:

1. **Используя DataFrame `df_seaborn_hue`:**
   - Построй скрипичную диаграмму, сравнивающую распределение **возраста** (**'Возраст'**) **по полу** (**'Пол'**). То есть, на оси X должен быть 'Пол', а на оси Y - 'Возраст'.
   - Измени **внутреннее отображение** на `'point'` (точки).
   - Измени **палитру цветов** на `'viridis'` (или любую другую на твой вкус).

- Попробуй разные значения параметра `scale` (`'area'` , `'count'` , `'width'`) и посмотри, как меняется ширина скрипок.

Выведи на экран все полученные скрипичные диаграммы. Попробуй интерпретировать результаты и сделать выводы о различиях в распределениях между категориями.

После выполнения заданий, обсудим результаты и подведем итоги по теме скрипичных диаграмм. Как тебе такой план? И есть ли вопросы по скрипичным диаграммам и их настройке? Не стесняйся спрашивать! 😉